

Pantallas LCD y Comunicación entre Circuitos Integrados

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Introducción a los Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

Resumen de la clase:

- 6.1 Temas para la semana 6.
- 6.2 Repaso: LCD.
- 6.3 Pines del LCD.
- 6.4 I2C.
- 6.5 Configuración I2C con CUBE MX.
- 6.6 Problema.

Temas para la semana 6

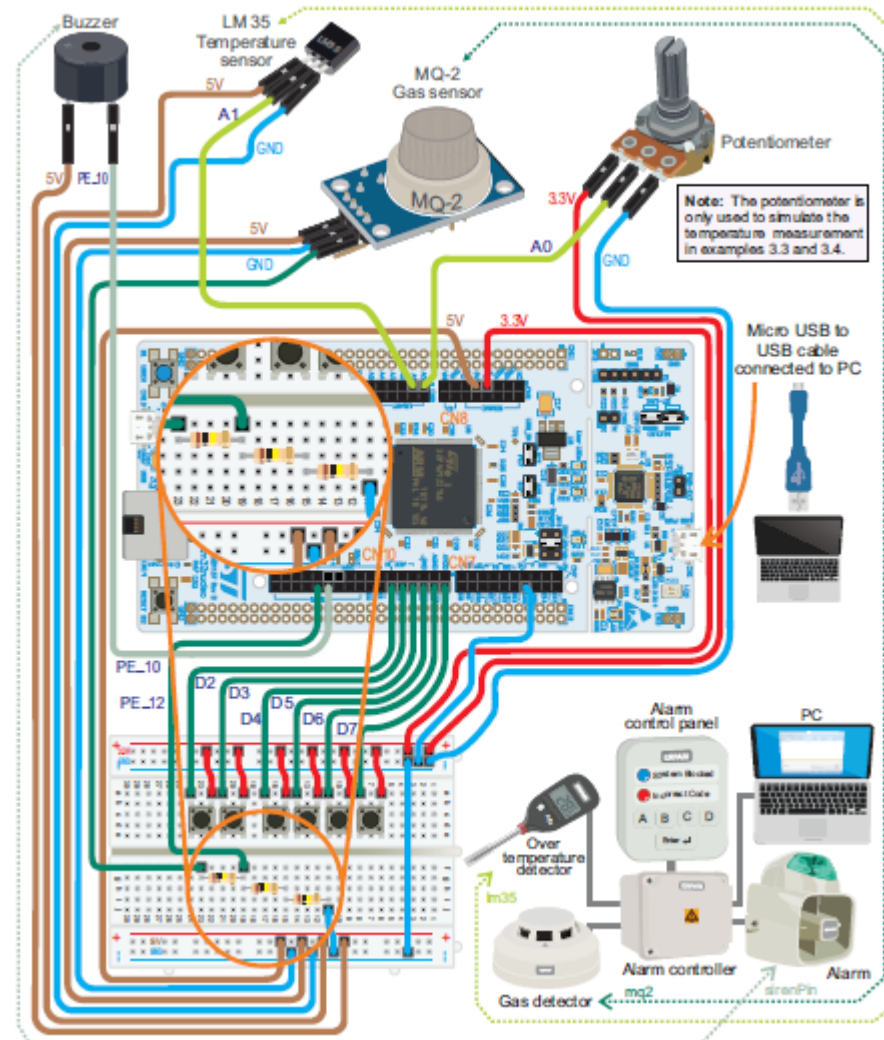


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

6.1 Temas para la semana 6.

6	Contenido de las clases de la semana 6. Teoría: Protocolos de comunicación entre circuitos integrados. <u>Display</u> para visualización de información. Características de las pantallas LCD. Práctica: Características de las pantallas LCD. Visualización de magnitudes en pantallas LCD.	Será informada por el docente a cargo.	1, 2, 3 y 4.
7	Contenido de las clases de la semana 7. Teoría: <u>Modularización</u> aplicada a la programación de sistemas embebidos. Módulo para drivers. Práctica: <u>Modularización</u>. Módulo para drivers. Envío de información de la placa NUCLEO-F446RE a <u>Node-RED</u> y conceptos de almacenamiento en base de datos.	Será informada por el docente a cargo.	1, 2, 3 y 4.
8	Exposición de los trabajos finales.		

Repaso: LCD

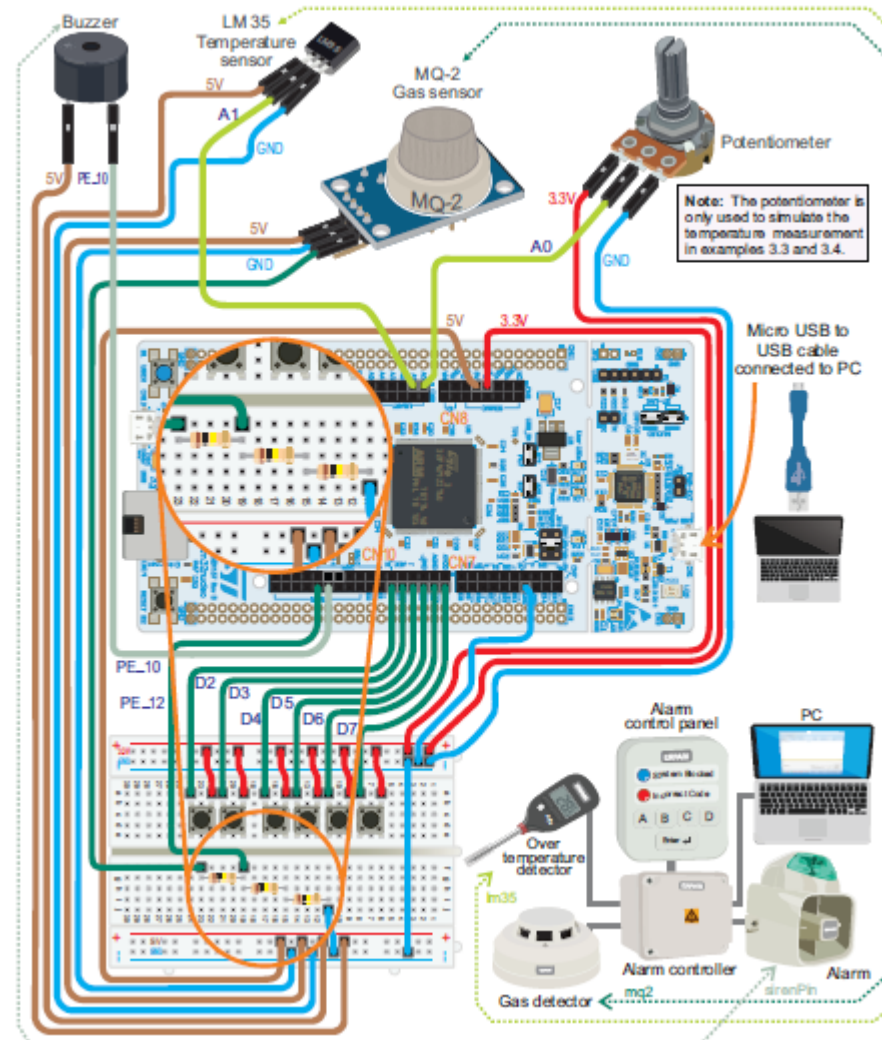


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

6.2 Repaso: LCD.



- Un LCD de 2x16 es una pantalla electrónica que puede mostrar 2 filas de 16 caracteres.
- Se utiliza en proyectos de electrónica para mostrar información en tiempo real, como menús, datos de sensores, relojes y niveles.
- Utiliza un chip controlador estándar como el HD44780.
- Estos módulos tienen 16 pines, por lo que se adapta un módulo I2C para la comunicación, reduciendo las conexiones a solo 4.

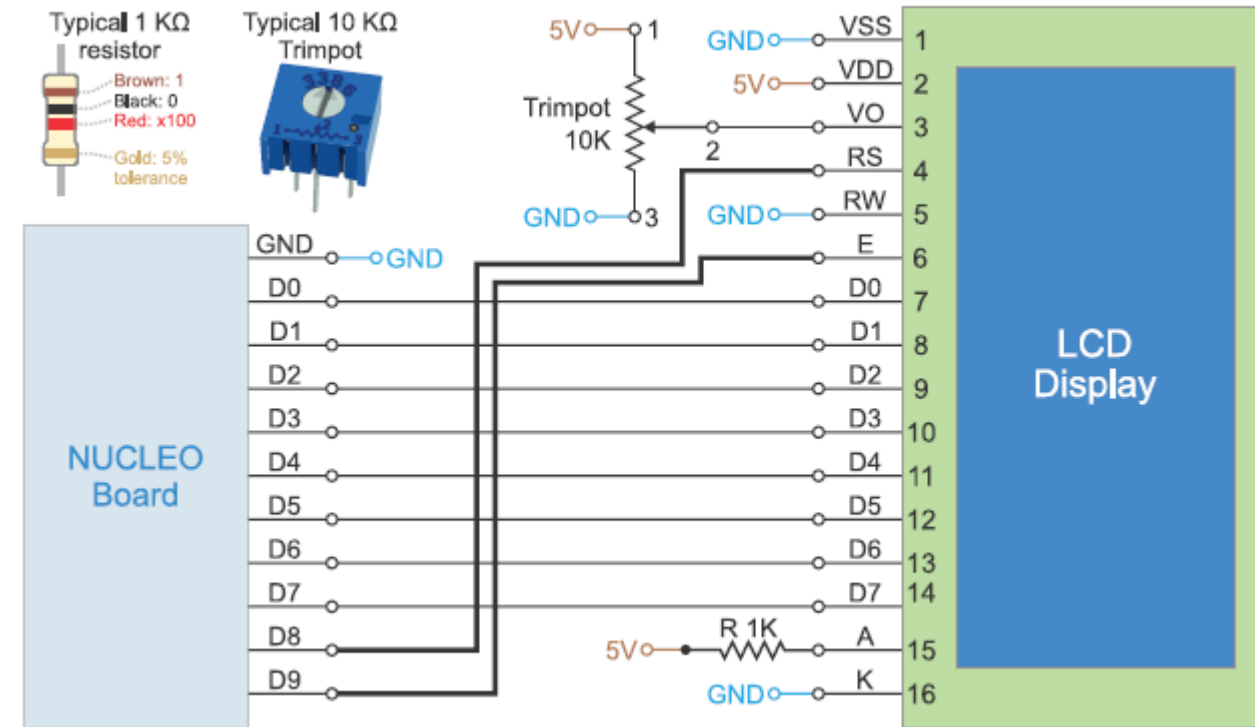
6.2 Repaso: LCD.

- En el display sólo se puede visualizar 16 caracteres por línea, sin embargo, se puede almacenar en total 40 por línea.
- Es el usuario indica los 16 caracteres que se van a visualizar.
- **ALIMENTACION:** La tensión nominal de alimentación es de 5V, con un consumo menor de 5mA.



6.2 Repaso: LCD.

- Conexiones:
- Las las GPIO D0-D9 que se utilizan para enviar comandos a la pantalla LCD.
- El contraste de la pantalla se ajusta mediante el potenciómetro.
- La resistencia de 1 k Ω conectada al pin A (ánodo) junto con el pin K (cátodo) se utiliza para alimentar la iluminación de la pantalla LCD.



6.2 Repaso: LCD.

- HD44780: controlador estándar de la industria diseñado para gestionar LCD de caracteres de matriz de puntos.
- Es compatible con pantallas de: 16x2, 20x4, 8x2 y 8x1.
- Cada carácter se representa típicamente en una matriz de 5x8 píxeles, donde la octava fila suele reservarse para mostrar el cursor.

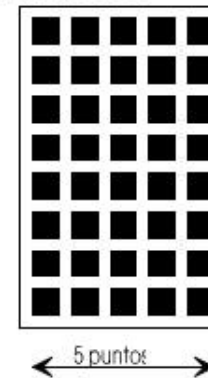


6.2 Repaso: LCD.

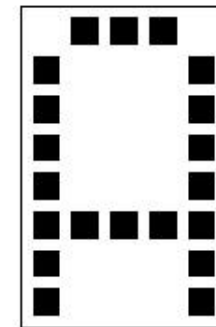
- El controlador utiliza el estándar ASCII para representar letras, dígitos y símbolos.
- Si bien los caracteres pueden ser de 5x8 pixels, la fuente son de 5x7 y la línea para cursor.
- Contiene una memoria CGRAM y DDRAM.



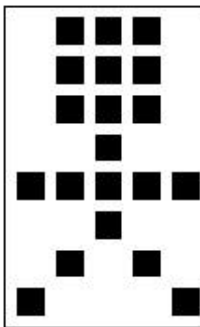
Matriz de representación de los caracteres



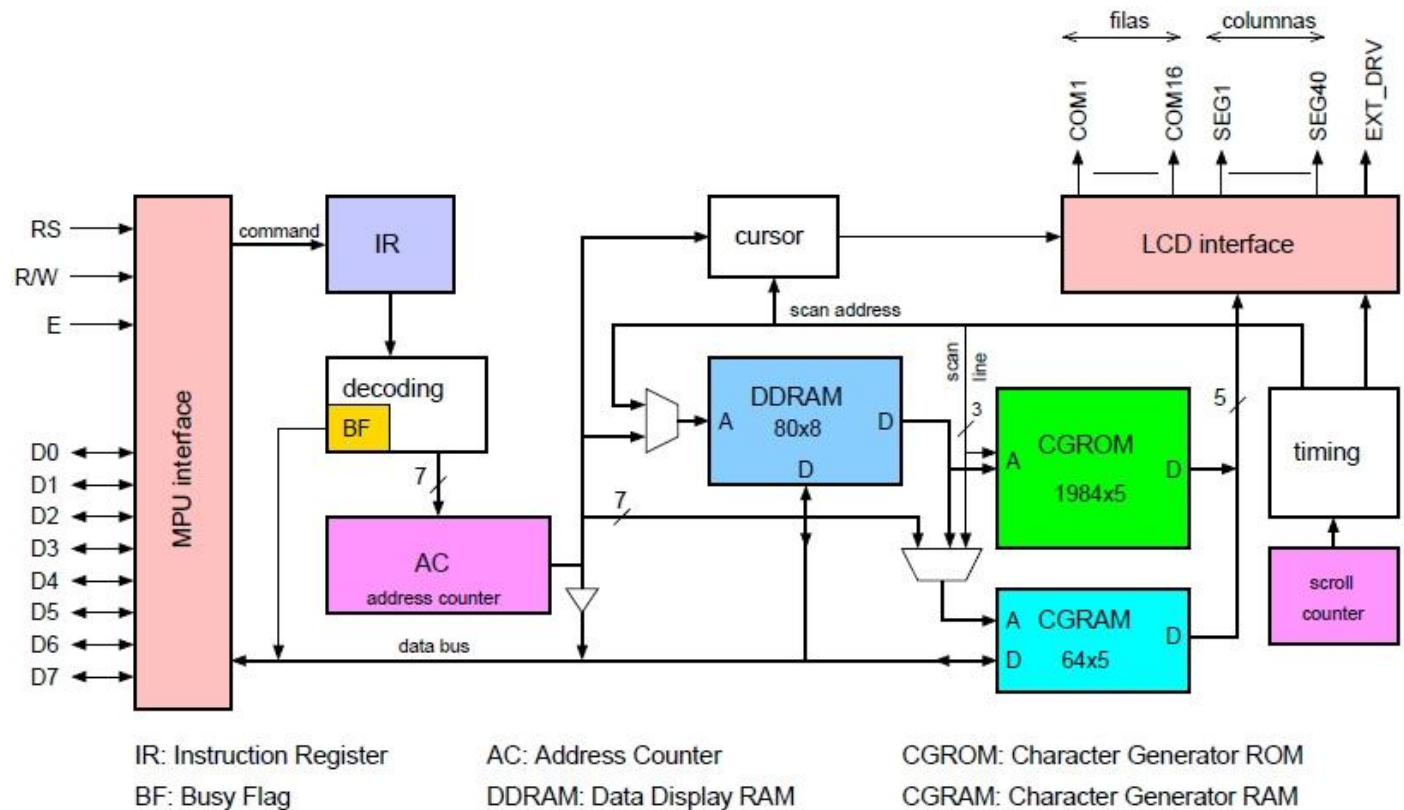
Carácter A



Carácter definido por el usuario



6.2 Repaso: LCD.



- CGRAM (Memoria de Acceso Aleatorio Generada a Medida): Permite al usuario definir hasta 8 caracteres personalizados.
- DDRAM (Display Data RAM): Almacena el código del carácter que debe mostrarse en cada posición física de la pantalla, corresponde a una ubicación específica en el display.

6.2 Repaso: LCD.

	0	32	40	48	56	64	72	80	88	96	104	112	120	160	168	176	184	192	200	208	216	224	232	240	248	
0	CG RAM (0)			(	0	8	G	H	P	X	`	h	P	X		イ	ー	ウ	ホ	タ	ミ	リ	ウ	ル	ロ	ク
1	CG RAM (1)		!	)	1	9	A	I	Q	Y	a	i	q	y		ア	フ	リ	チ	ム	ル	サ	リ	ウ	ク	
2	CG RAM (2)		"	*	2	:	B	J	R	Z	b	j	r	z		エ	イ	コ	ン	ツ	ク	レ	セ	イ	ロ	フ
3	CG RAM (3)		#	+	3	;	C	K	S	L	c	k	s	l		オ	ウ	サ	セ	テ	モ	ロ	セ	ウ	ク	フ
4	CG RAM (4)		\$	,	4	<	D	L	T	¥	d	l	t	¥		カ	エ	ソ	フ	ト	ホ	ウ	ク	ロ	フ	
5	CG RAM (5)		%	-	5	=	E	N	U	J	e	n	u	j		ユ	オ	ス	セ	ナ	ユ	セ	ト	ウ	ク	フ
6	CG RAM (6)		&	.	6	>	F	N	V	^	f	n	v	^		ヨ	カ	セ	ホ	ニ	ヨ	セ	ロ	ン	ク	フ
7	CG RAM (7)		^	/	7	?	G	O	U	_	g	o	u	_		ツ	チ	リ	マ	ス	ラ	ア	ロ	ン	ク	フ

- El código correspondiente a cada carácter se obtiene de la suma de los valores de fila y columna.
- Por ejemplo: el carácter “A” (intersección 64 y 1) tiene un código 65.

6.2 Repaso: LCD.

	0	32	40	48	56	64	72	80	88	96	104	112	120	160	168	176	184	192	200	208	216	224	232	240	248
0 CG RAM (0)			C	O	S	G	H	P	X	\	h	P	X		イ	ー	ウ	ホ	タ	ミ	リ	ウ	ル	ロ	ク
1 CG RAM (1)		!	)	1	9	A	I	O	Y	a	i	9	y		ウ	ア	テ	リ	チ	ム	ル	ア	ー	ウ	
2 CG RAM (2)		"	*	2	:	B	J	R	Z	b	j	r	z		エ	イ	コ	ン	ツ	×	レ	エ	イ	ロ	キ
3 CG RAM (3)		#	+	3	:	C	K	S	L	c	k	s	l		ホ	ウ	サ	セ	テ	モ	ロ	エ	ウ	ル	
4 CG RAM (4)		\$	,	4	<	D	L	T	¥	d	l	t	¥		ホ	エ	シ	フ	ト	フ	ウ	ホ	ロ	キ	
5 CG RAM (5)		%	-	5	=	E	N	U	J	e	n	u	j		ユ	オ	ス	ン	タ	ユ	ン	ロ	キ	ウ	÷
6 CG RAM (6)		&	.	6	>	F	N	V	^	f	n	v	^		ヨ	カ	セ	ホ	ニ	ヨ	ン	ロ	キ	ウ	
7 CG RAM (7)		^	/	7	?	G	O	U	_	g	o	u	_		ア	ッ	チ	リ	ス	ス	ラ	ア	ロ	キ	■

- Se pueden representar 256 caracteres diferentes.
- 240 caracteres están grabados dentro del LCD y 8 (del 0 al 7) caracteres que pueden ser definidos por el usuario.

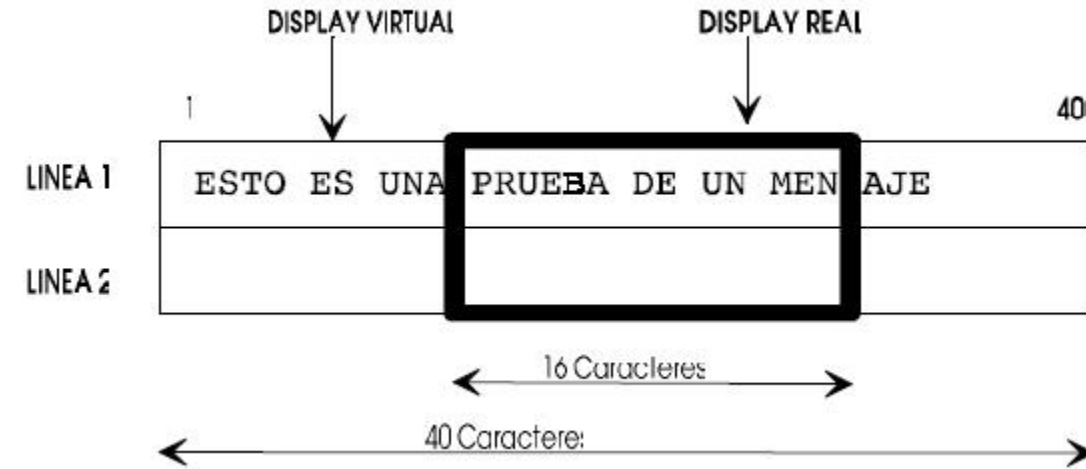
6.2 Repaso: LCD.

	0	32	40	48	56	64	72	80	88	96	104	112	120	160	168	176	184	192	200	208	216	224	232	240	248	
0	CG RAM (0)			(	0	8	9	H	P	X	\	h	P	X		イ	ー	ウ	本	タ	三	リ	α	γ	θ	Σ
1	CG RAM (1)	!	)	1	9	A	I	Q	Y	a	i	9	y		ウ	ア	テ	リ	チ	△	ル	ä	¹	au		
2	CG RAM (2)	"	*	2	:	B	J	R	Z	b	j	r	z	Γ	エ	イ	コ	ン	ツ	×	レ	θ	ι	θ	τ	
3	CG RAM (3)	#	+	3	:	C	K	S	L	c	k	s	l	┐	★	ウ	サ	セ	テ	モ	ロ	ε	×	α	π	
4	CG RAM (4)	\$	,	4	<	D	L	T	¥	d	l	t	┐	ノ	ハ	エ	シ	フ	ト	フ	ワ	Φ	α	π		
5	CG RAM (5)	½	-	5	=	E	N	U	J	e	n	u	┐	・	ユ	オ	ス	△	★	1	2	α	±	ü	÷	
6	CG RAM (6)	8	.	6	>	F	N	V	^	f	n	v	→	ヲ	ヨ	カ	セ	ホ	ニ	ヨ	°	o	ñ	Σ		
7	CG RAM (7)	7	/	7	?	G	O	U	_	9	o	w	+	ア	ッ	†	リ	ス	ス	ラ	°	a	ö	π	■	

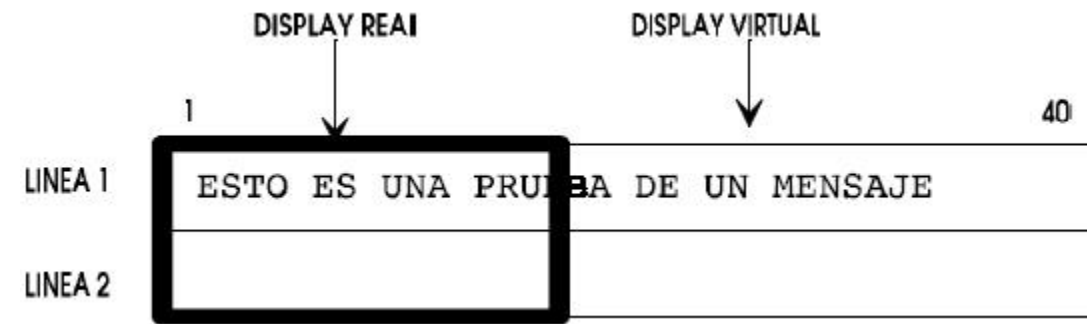
- Los siguientes ocho (del 8 al 15) no se muestran en la Tabla.
- Los caracteres del 16 al 31 y del 128 al 159 no se incluyen, pueden variar.

6.2 Repaso: LCD.

- En la memoria **DDRAM** se almacenan los caracteres que se visualizan y los que no.
- El display almacena en esta memoria dos líneas de 40 caracteres pero sólo se visualizan 2 líneas de 16 caracteres.
- La **DDRAM** tiene un tamaño de $2 \times 40 = 80$ bytes.



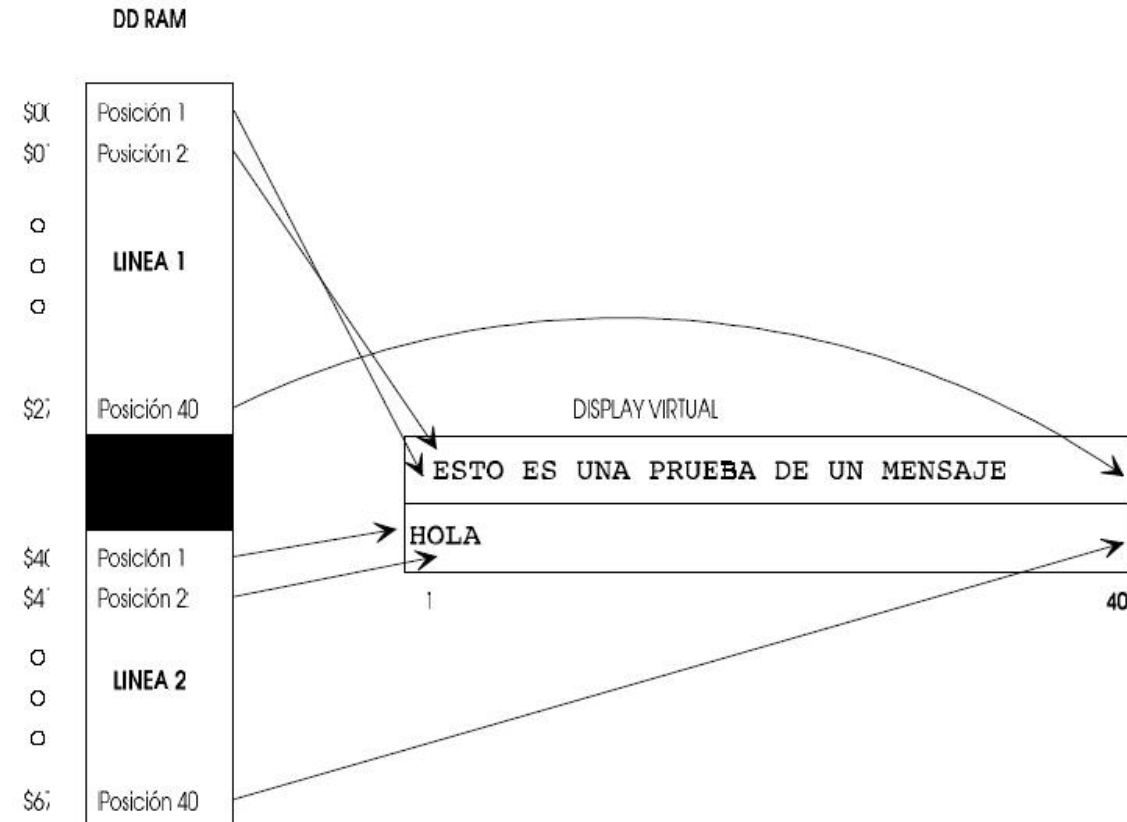
6.2 Repaso: LCD.



- Se puede pensar en el display como un *display virtual* constituido por dos líneas de 40 caracteres cada una.
- Para localizar los elementos dentro del *display virtual* se va a utilizar un par de coordenadas (x,y) donde x representa la posición horizontal (comprendida entre 1-40) y representa la línea (1-2).
- El display real es una *ventana* en la que se visualizan dos líneas de 16 caracteres.

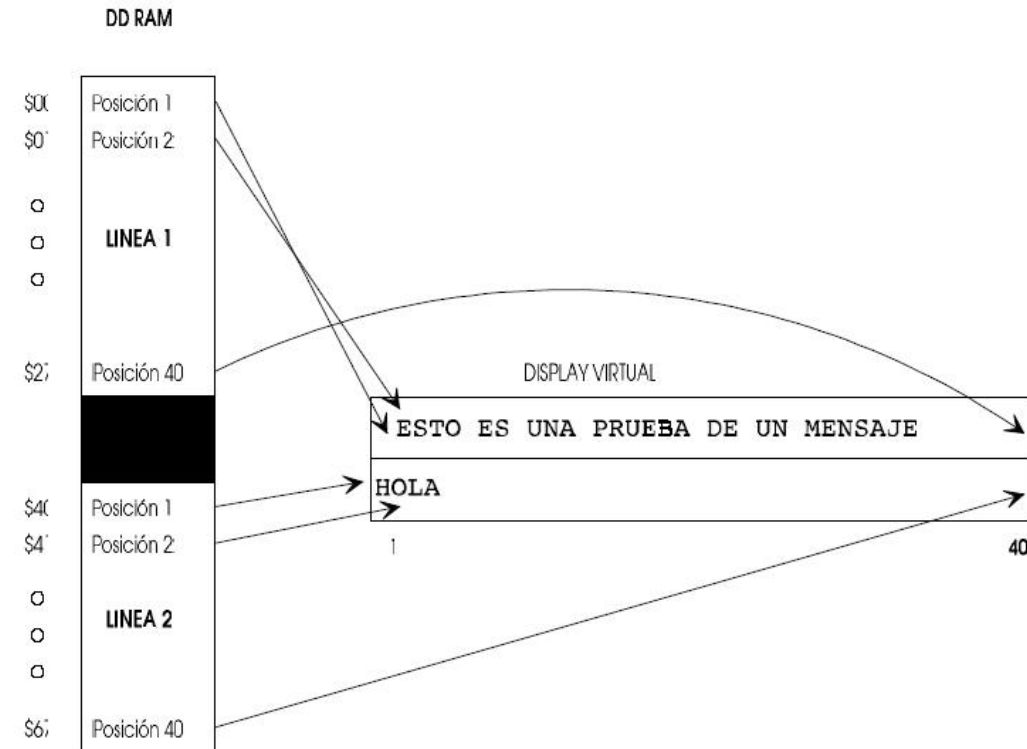
6.2 Repaso: LCD.

- **DDRAM**: está constituida por dos bloques de 40 bytes.
- El primer corresponde a los 40 caracteres de la línea 1 y el segundo bloque con la segunda línea.
- En la figura se han representado las direcciones en hexadecimal.
- Las direcciones \$00-\$27 están asociadas a las posiciones (1,1)-(40,1) del display virtual y las direcciones \$40-\$67 a las posiciones (1,2)-(40,2).



6.2 Repaso: LCD.

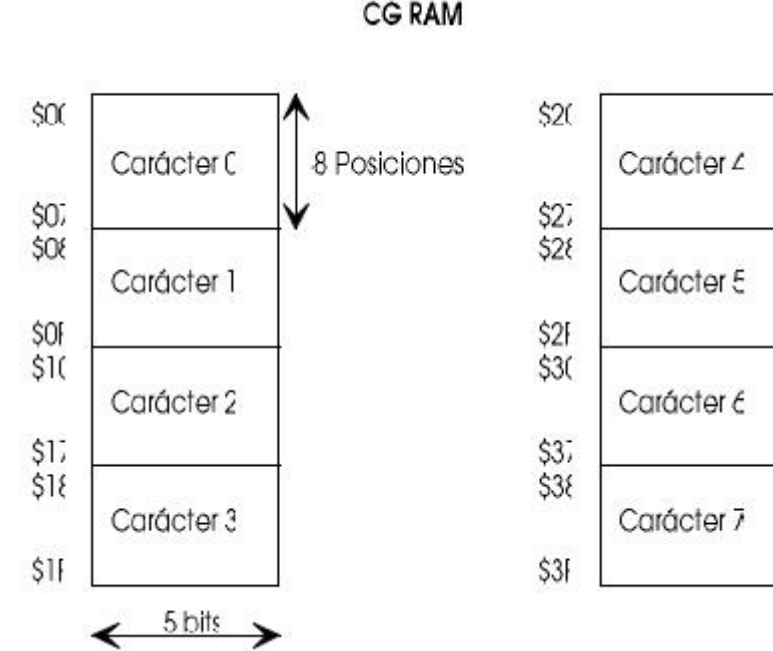
- Los bloques de memoria asociados a la línea 1 y 2 no son contiguos.



- Las operaciones de escritura en el display, en realidad son operaciones de escritura en la memoria DD RAM.
- En las posiciones (1,1)-(16,1) y (1,2)-(16,2) son visibles.
- Si se envía un carácter a cualquiera de las otras posiciones no será visible.

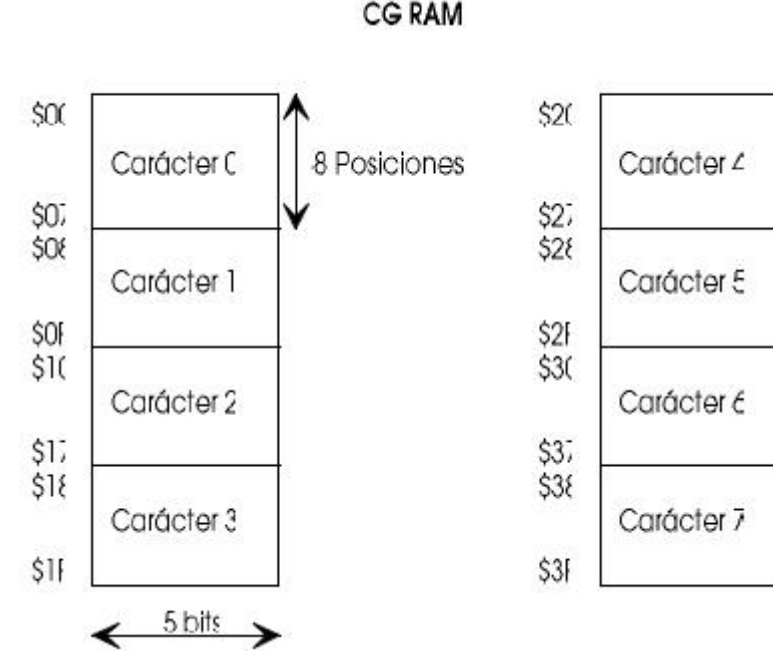
6.2 Repaso: LCD.

- Memoria **CGRAM (Character Generator RAM)**;
- La CGRAM es la memoria que contiene los caracteres definibles por el usuario.
- Está formada por 64 posiciones, con direcciones \$00-\$3F, cada posición es de 5 bits.
- La memoria está dividida en 8 bloques, correspondiendo cada bloque a un carácter definible por el usuario.



6.2 Repaso: LCD.

- Memoria **CGRAM (Character Generator RAM)**;



- Por ello el usuario puede definir como máximo 8 caracteres, cuyos códigos van del 0 al 7.
- Para definir un carácter y asignarle por ejemplo el código 0 habrá que almacenar en las posiciones \$00-\$07 los valores binarios de las 8 filas del carácter del usuario.

Pines del LCD

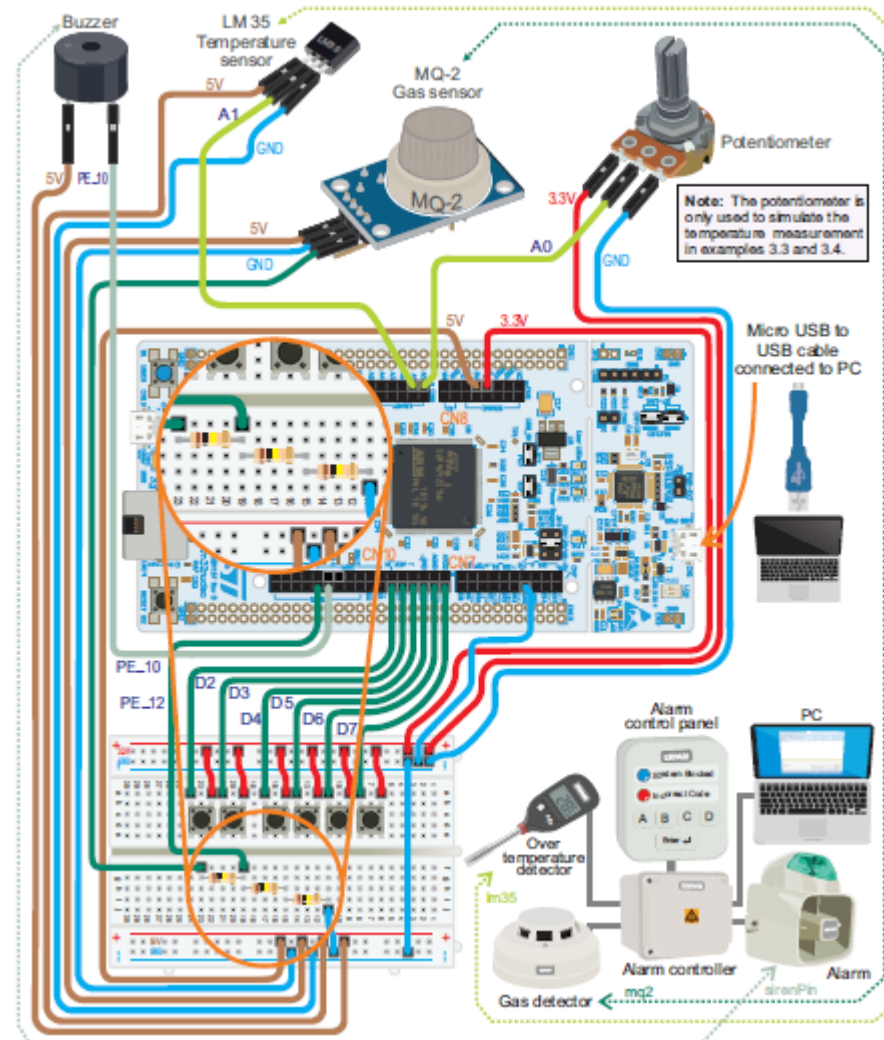
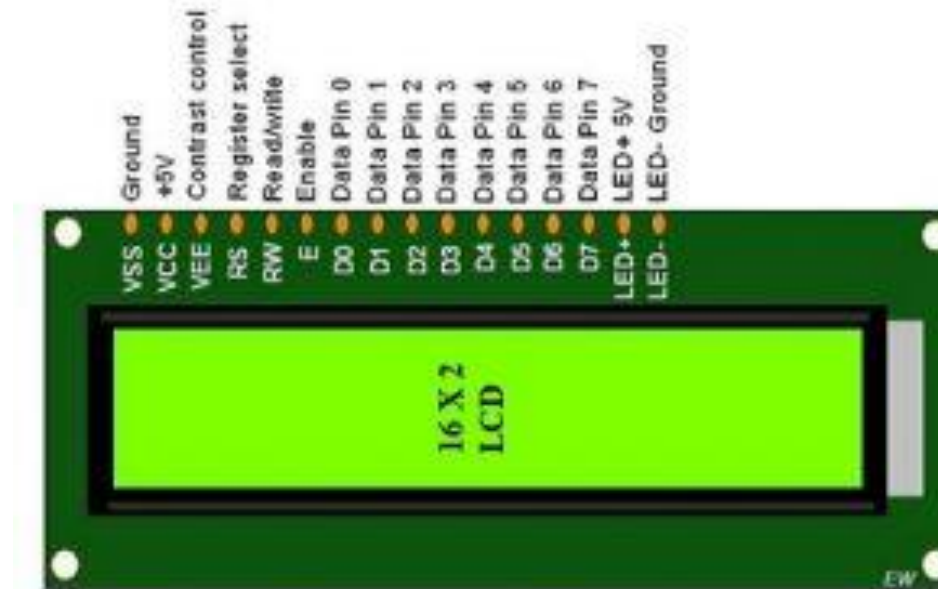


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

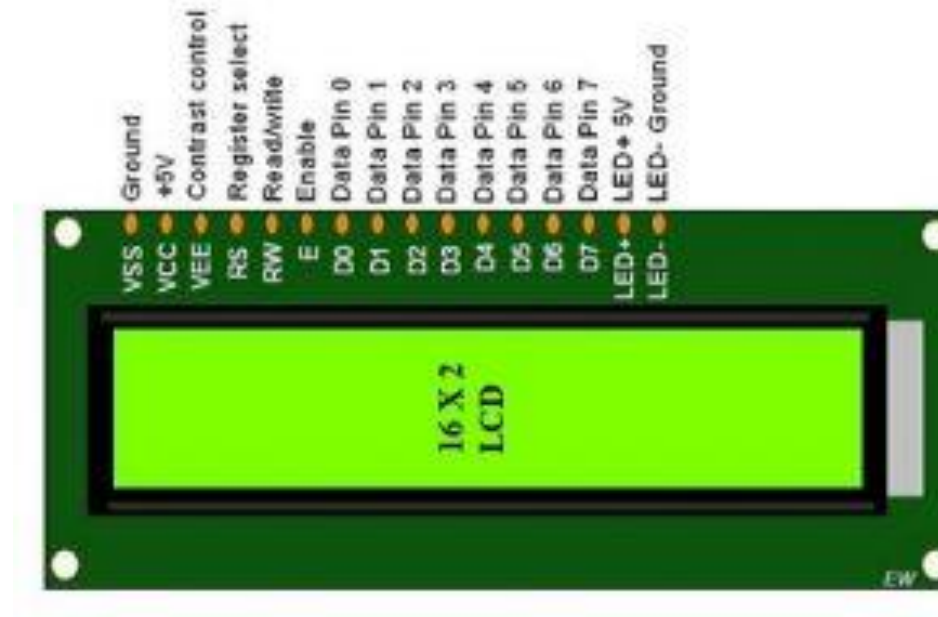
<https://www.arm.com/resources/education/books/designing-embedded-systems>

6.3 Pines del LCD.



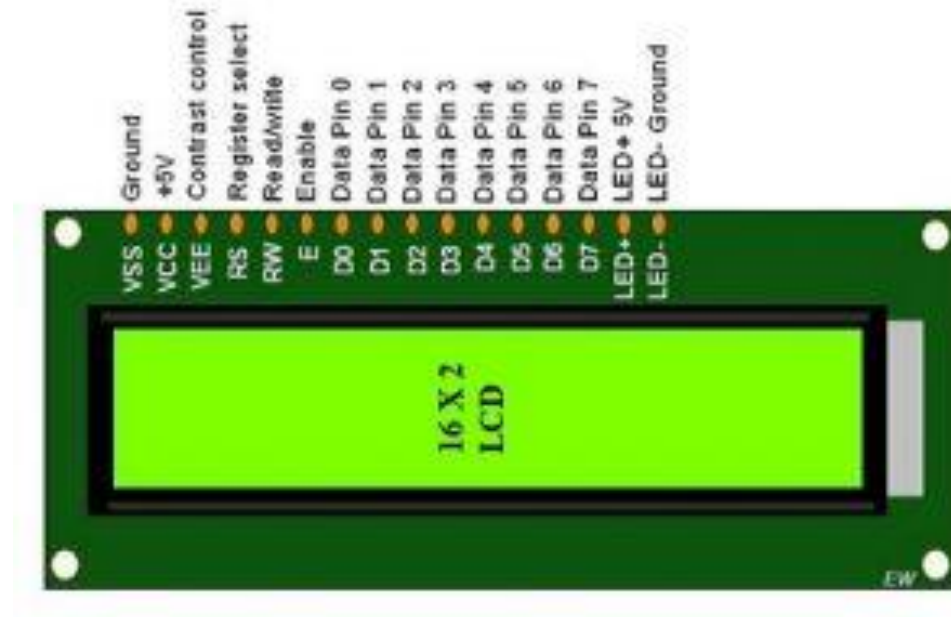
- RS (Register Select - Selector de Registro): determina si la información enviada al display es un comando o un dato (carácter).
- **Nivel lógico bajo:** para escribir en el registro de instrucciones (por ejemplo, para limpiar la pantalla o mover el cursor)
- **Nivel lógico alto:** para escribir en el registro de datos (el código ASCII del carácter que se desea mostrar).

6.3 Pines del LCD.



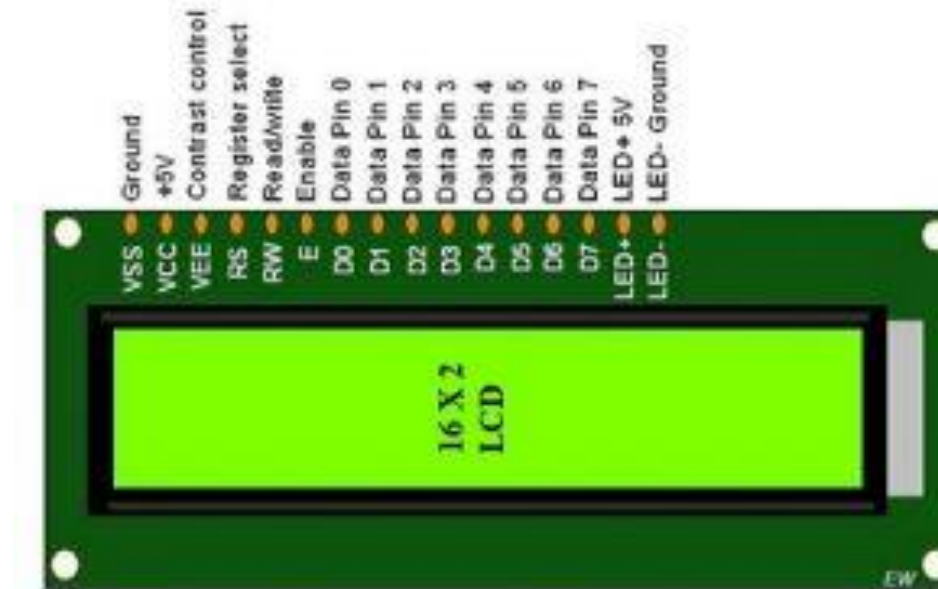
- RW (Read/Write - Lectura/Escritura): selecciona la dirección de la transferencia de datos entre el microcontrolador y el LCD.
- **Nivel bajo:** indica una operación de escritura.
- **Nivel alto:** indica una de lectura. En la práctica común este pin suele conectarse directamente a GND (0).

6.3 Pines del LCD.



- E (Enable - Habilitación): inicia la transferencia de la información.
- Señal: El funcionamiento consiste en establecer primero los estados de los otros pines (RS, RW y bus de datos) y luego aplicar un pulso positivo que debe durar al menos 1 μ s.
- La información es capturada específicamente durante el flanco de bajada de este pulso.

6.3 Pines del LCD.

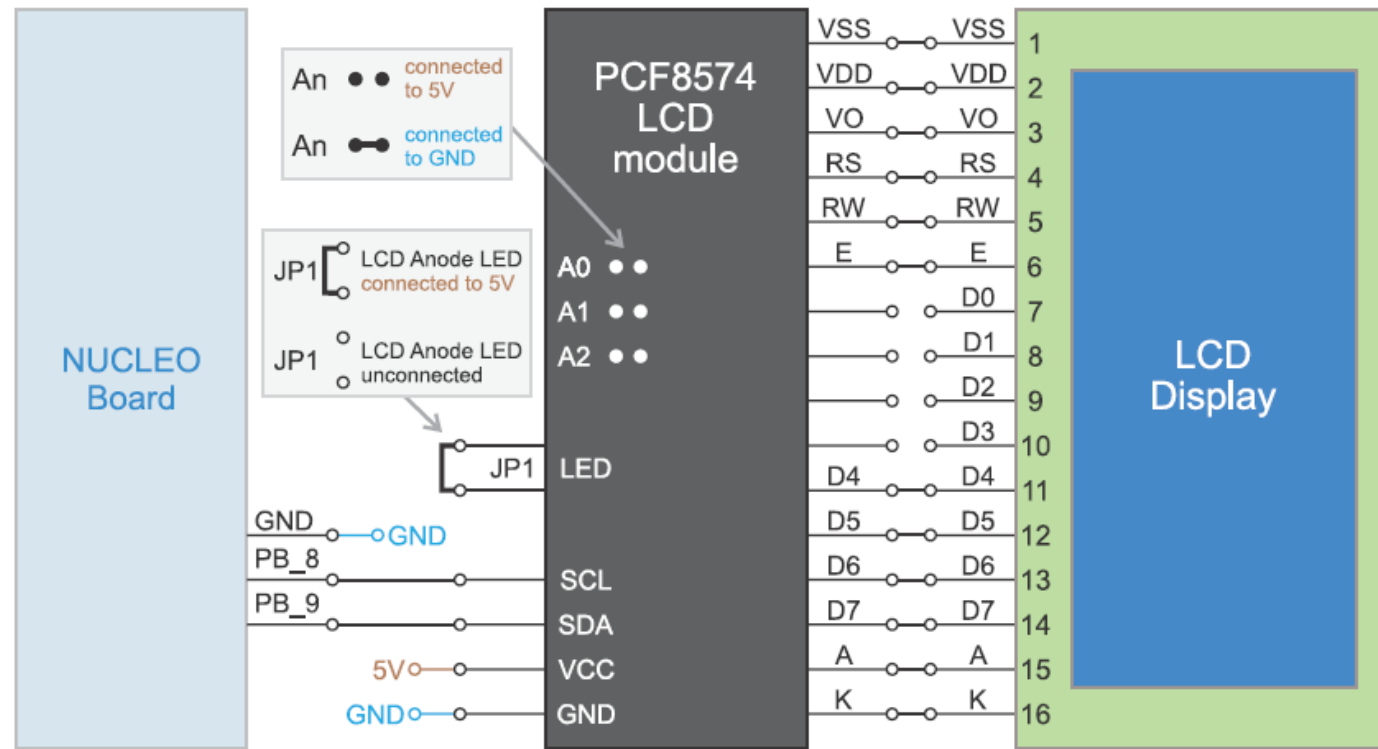


- Bus de Datos (D0 a D7): los pines encargados de transportar el byte de información, ya sea una instrucción o un carácter.
- Modo de 8 bits (D0-D7): para enviar el byte completo en una sola operación de pulso en el pin E.
- Modo de 4 bits (D4 a D7): se envían los 4 bits más significativos y luego los 4 menos significativos, requiriendo dos pulsos en E.

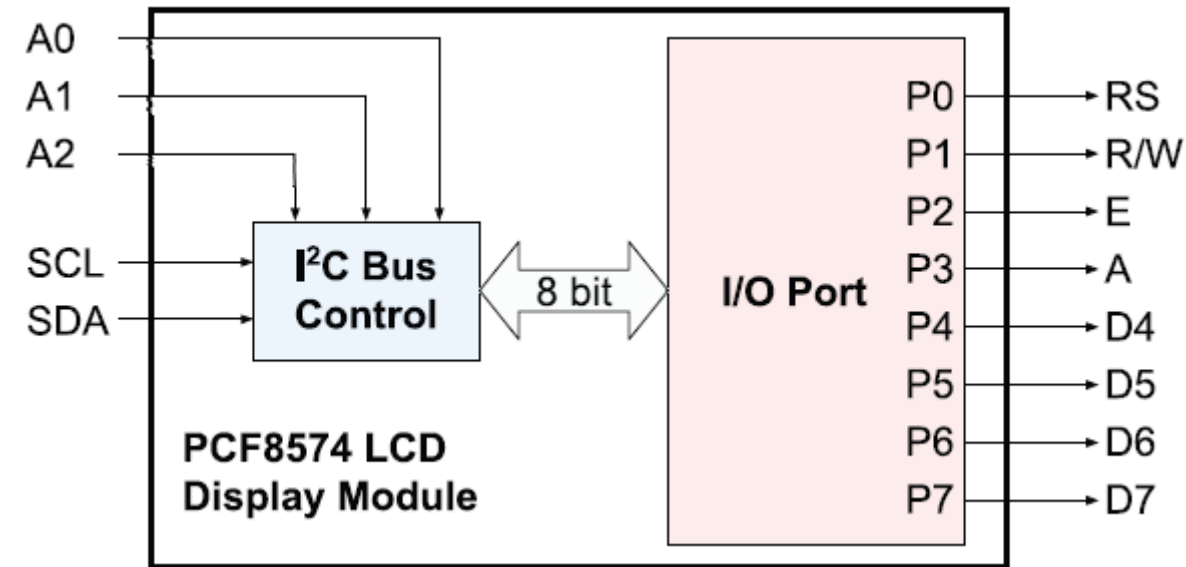
6.3 Pines del LCD.

Conexión de una pantalla LCD mediante el bus I2C.

- Se realiza una configuración que utiliza menos cables para conectar la pantalla LCD.
- Se utiliza un módulo basado en el expansor de E/S de 8 bits PCF8574 para el bus I2C.
- Este módulo se controlan mediante una conexión de dos hilos.



6.3 Pines del LCD.



- El estado de los pines D7, D6, D5, D4, A, E, R/W y RS se controla mediante el bus I2C.
- Los pines A0, A1 y A2 se utilizan para configurar la dirección del módulo PCF8574, como se muestra en la Tabla.
- Si estos pines se dejan sin conectar, estarán en estado alto debido a las resistencias pull-up del módulo, y se configurará la dirección de escritura de 8 bits 78 del bus I2C.

I2C

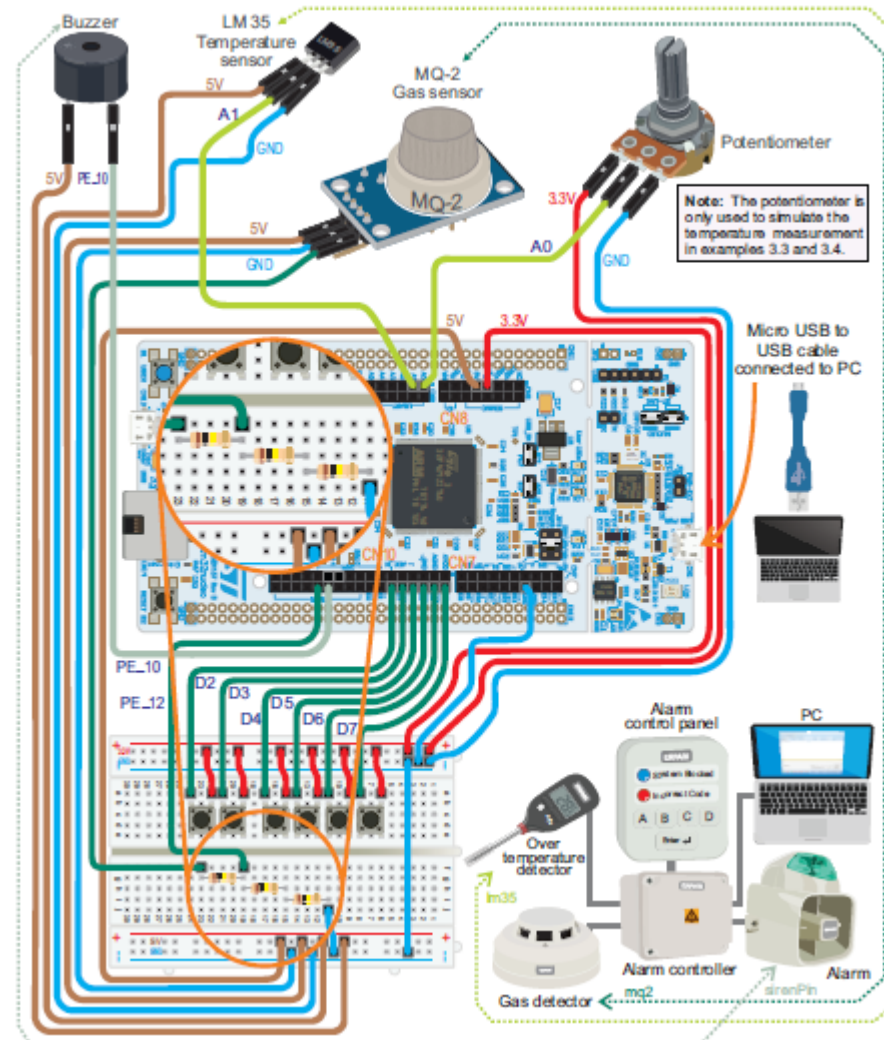
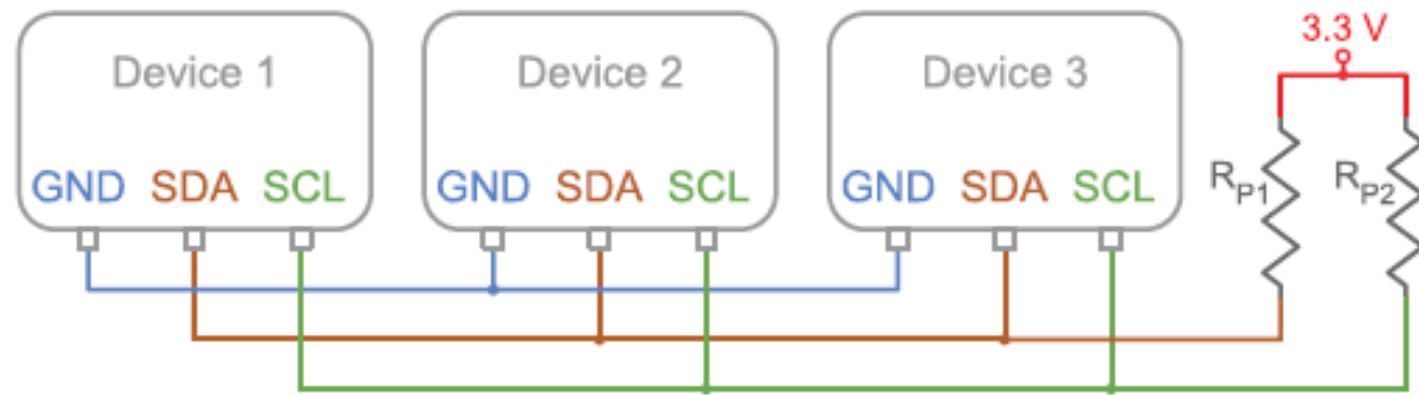


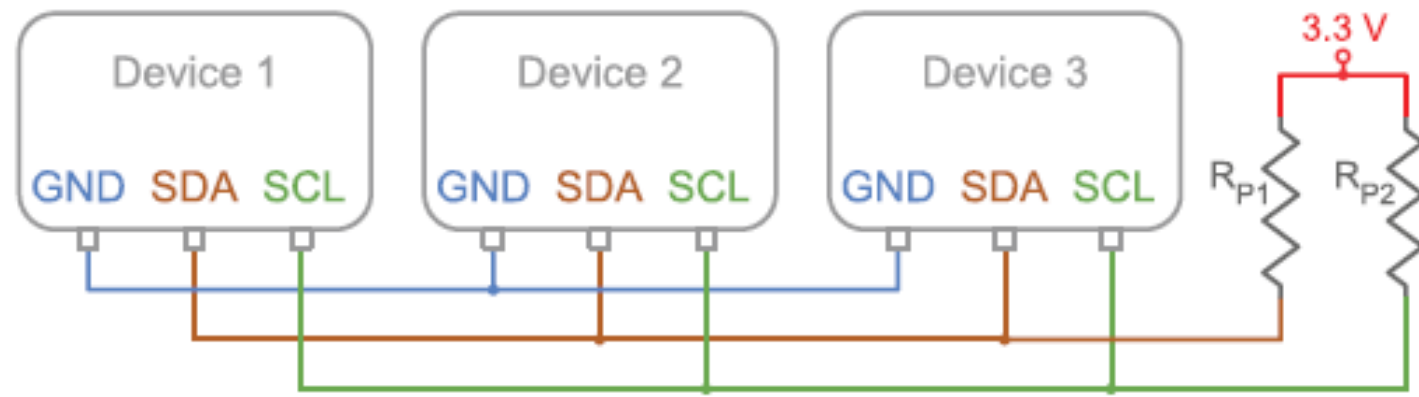
Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.
<https://www.arm.com/resources/education/books/designing-embedded-systems>

6.4 I2C



- El protocolo I2C (Inter-Integrated Circuit) es un bus de comunicación serie síncrono diseñado para conectar circuitos integrados.
- Utiliza dos líneas: SDA (Serial Data Line) para los datos y SCL (Serial Clock Line) para la señal de reloj.
- El I2C permite conectar hasta 127 dispositivos compartiendo los mismos dos cables.
- Resistencias Pull-up: Debido a que las salidas SDA y SCL son de tipo colector abierto (o drenador abierto), es necesario.

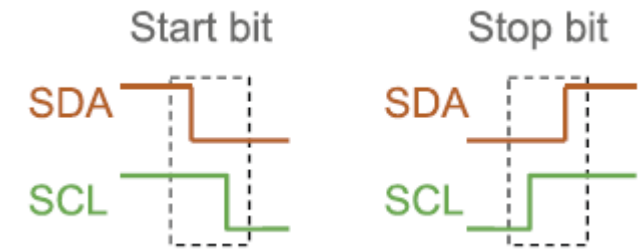
6.4 I2C



Roles y Direccionamiento:

- La comunicación la inicia un dispositivo Maestro que genera la señal de reloj y gestiona el flujo de datos.
- Los demás actúan como Esclavos y responden cuando se les llama.
- Cada dispositivo en el bus tiene una dirección única (7 bits).
- El Maestro envía esta dirección al inicio de la trama para que solo el receptor deseado responda.

6.4 I2C.

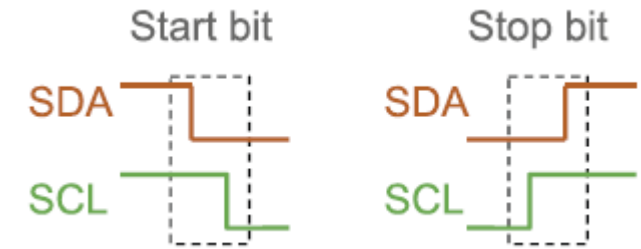


Protocolo de Comunicación:



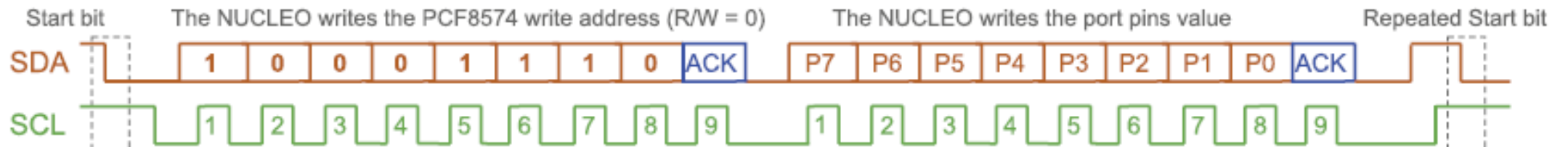
- Condición de Inicio (Start): El Maestro pone la línea SDA en bajo mientras SCL está en alto.
- El Maestro envía los 7 bits de dirección del esclavo seguidos de un bit R/W (lectura o escritura).
- Bit ACK (Acknowledge): Tras recibir la dirección, el dispositivo de destino debe poner SDA en bajo durante el noveno pulso de reloj para confirmar la recepción.

6.4 I2C.



Protocolo de Comunicación:

- Transferencia de Datos: Los datos se envían byte a byte, cada uno seguido de su correspondiente bit de ACK.
- Condición de Parada (Stop): El Manager pone SDA en alto mientras SCL está en alto, liberando el bus.



Configuración I2C con CUBE MX.

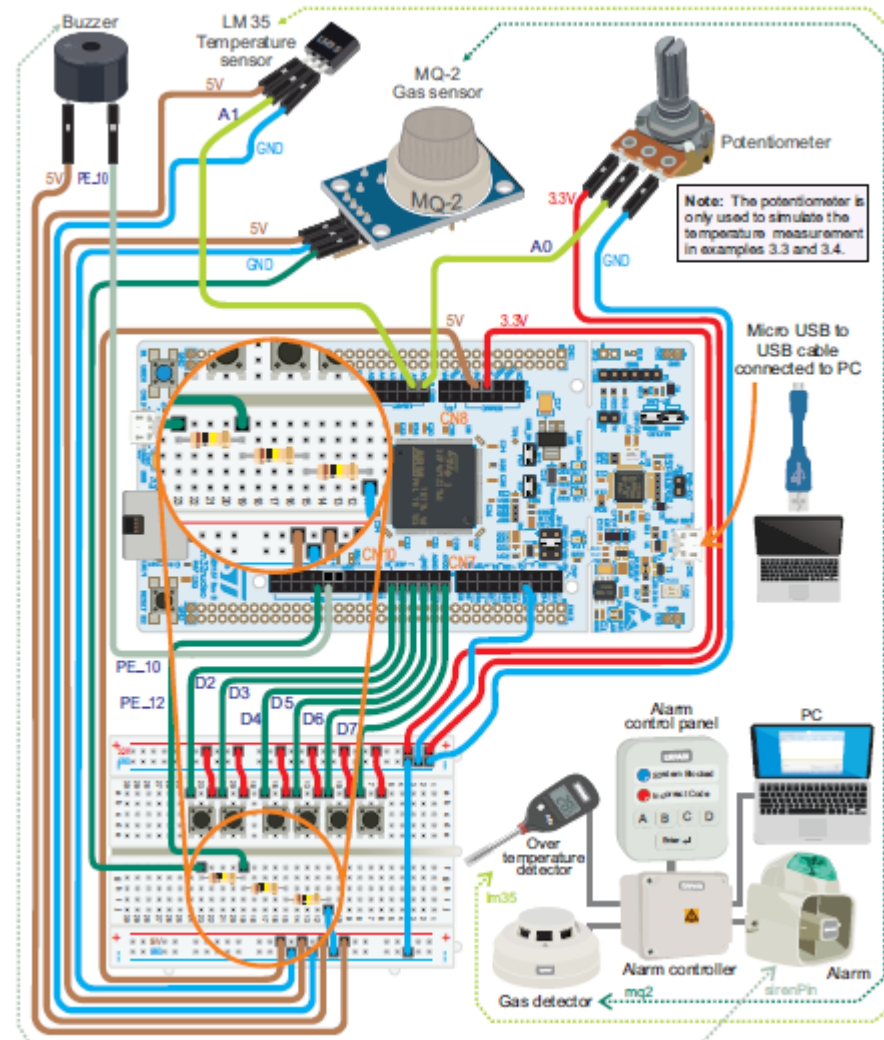


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

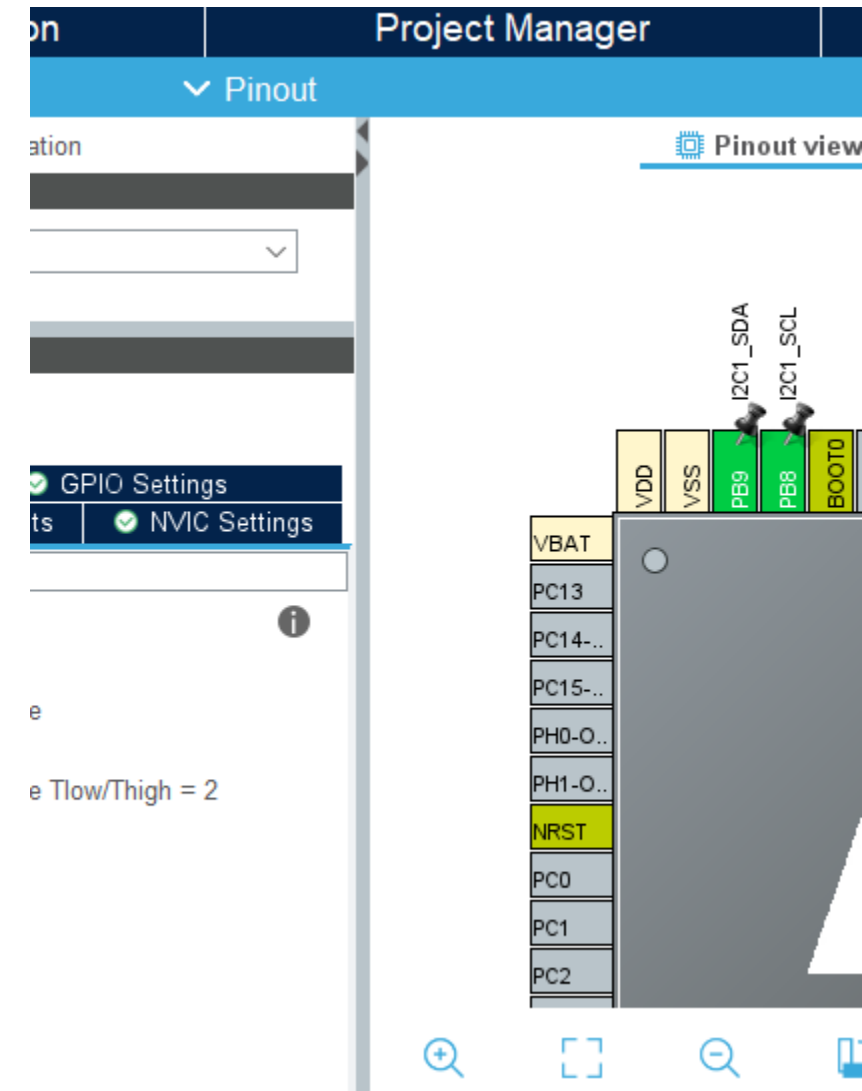
6.5 Configuración I2C con CUBE MX.

The screenshot displays the STM32CubeMX Pinout & Configuration window. The interface is divided into several sections:

- Pinout & Configuration** (Active tab):
 - Categories**: A-Z, Connectivity.
 - Connectivity**: A list of peripheral categories including CAN1, CAN2, FMPI2C1, **I2C1** (selected), I2C2, I2C3, QUADSPI, SDIO, SPI1, SPI2, SPI3, UART4, UART5, USART1, USART2, USART3, USART6, USB_OTG_FS, and USB_OTG_HS.
- I2C1 Mode and Configuration**:
 - Mode**: I2C.
 - Configuration**:
 - Reset Configuration** button.
 - DMA Settings**, **GPIO Settings**, **Parameter Settings** (selected), **User Constants**, and **NVIC Settings** tabs.
 - Configure the below parameters :**
 - Master Features**:
 - I2C Speed Mode: Fast Mode
 - I2C Clock Speed (Hz): 400000
 - Fast Mode Duty Cycle: Duty cycle Tlow/Thigh = 2
 - Slave Features**:
 - Clock No Stretch Mode: Disabled
 - Primary Address Length: 7-bit
 - Dual Address Acknowledgment: Disabled
 - Primary slave address: 0
 - General Call address disabled: Disabled
- Pinout** (Inactive tab):
 - Pinout view**: A diagram of the microcontroller pinout. The pins are labeled: VBAT, VDD, VSS, PB9 (I2C1_SDA), PB8 (I2C1_SCL), BOOT0, PC13, PC14, PC15, PH0-O, PH1-O, NRST, PC0, PC1, and PC2.

6.5 Configuración I2C con CUBE MX.

- Dos puntos clave:
- Se puede elegir Standard Mode y Fast Mode: La diferencia principal radica en la velocidad máxima de transmisión de datos (100kHz o 400KHz).
- Cambiar los pines SDA y SCL.



Problemas.

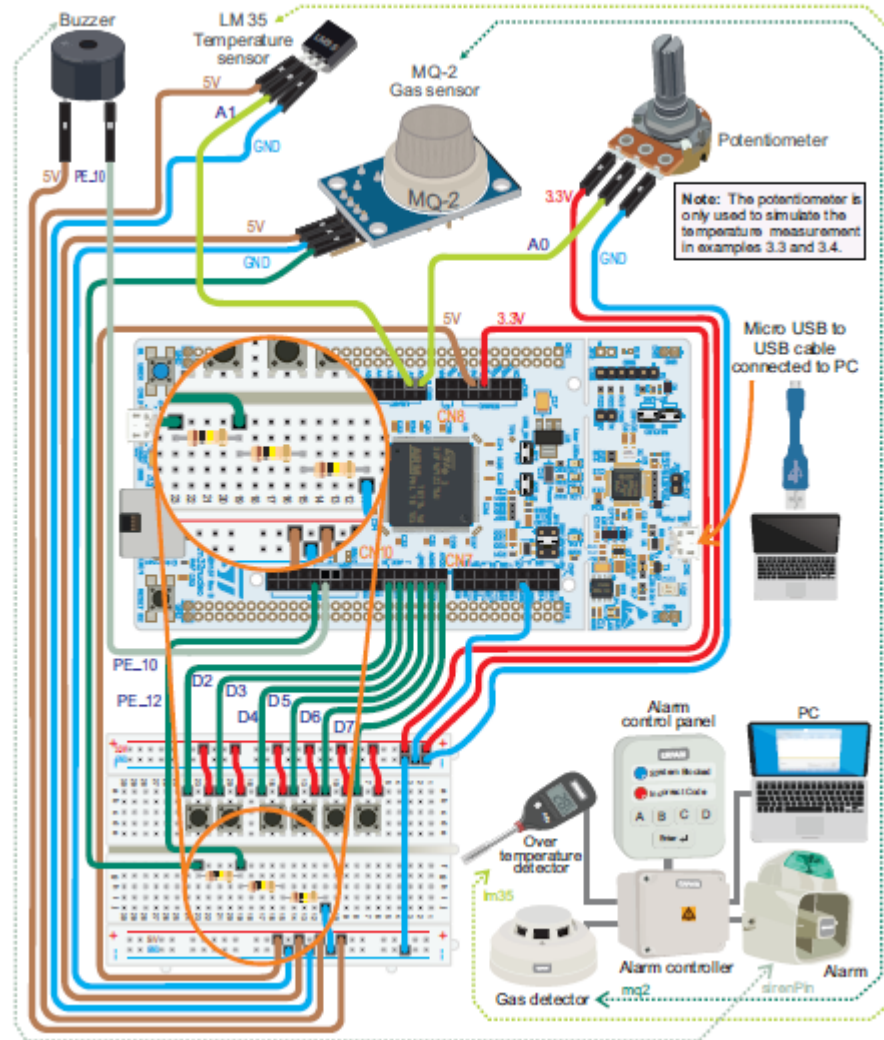


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.
<https://www.arm.com/resources/education/books/designing-embedded-systems>

6.6 Problema.

- Analice los archivos i2c_lcd.c e i2c_lcd.h.

```
#ifndef I2C_LCD_H
#define I2C_LCD_H
#include <stdint.h>
#include "i2c.h"
#define SLAVE_ADDRESS_LCD 0x27
#define LCD_CMD_CLEAR_DISPLAY 0x01
#define LCD_CMD_RETURN_HOME 0x02
#define LCD_CMD_ENTRY_MODE_SET 0x06
#define LCD_CMD_DISPLAY_ON 0x0C
#define LCD_CMD_DISPLAY_OFF 0x08
#define LCD_CMD_FUNCTION_SET 0x28
#define LCD_CMD_SET_CURSOR 0x80
#define LCD_CMD_INIT_8_BIT_MODE 0x30
#define LCD_CMD_INIT_4_BIT_MODE 0x20
```

```
void lcd_init(void);
HAL_StatusTypeDef lcd_send_cmd(uint8_t cmd);
HAL_StatusTypeDef lcd_send_data(uint8_t data);
void lcd_send_string(char *str);
void lcd_put_cursor(int row, int col);
void lcd_clear(void);
uint8_t scan_i2c_address(void);
#endif
```

6.6 Problema.

Realizar una conversión AD y mostrar la magnitud de un float en el LCD.

```
#include "main.h"
#include "adc.h"
#include "i2c.h"
#include "gpio.h"
#include "i2c_lcd.h"
uint32_t adc_raw;
float Temp;
void SystemClock_Config(void);
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_I2C1_Init();
    lcd_init();
```

```
while (1){
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){
        adc_raw = HAL_ADC_GetValue(&hadc1);
        Temp = adc_raw * 3.3 / (4095.0*0.01);
    }
    lcd_put_cursor(0,0);
    lcd_send_string("Temp");
    lcd_put_cursor(0,10);
    lcd_send_string(" ");
    lcd_put_cursor(0,10);
    lcd_send_float(Temp, 1);
    lcd_put_cursor(1,0); lcd_send_string("STM32F446RE");
    HAL_Delay(1000); }
}
```

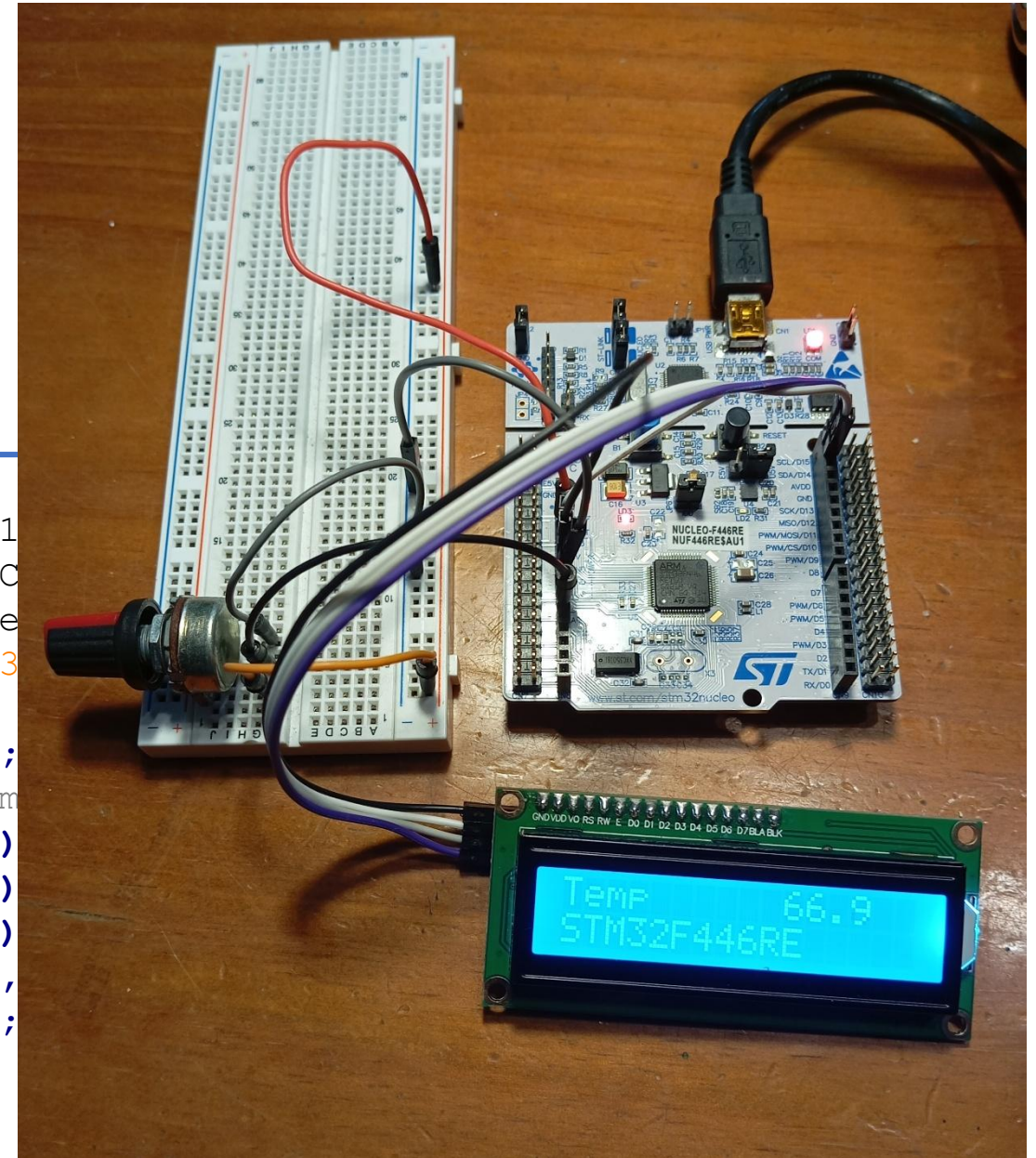
6.6 Problema.

Realizar una conversión AD y mostrar la magnitud de un float en el LCD.

RESULTADOS:

```
#include "main.h"
#include "adc.h"
#include "i2c.h"
#include "gpio.h"
#include "i2c_lcd.h"
uint32_t adc_raw;
float Temp;
void SystemClock_Config(void);
int main(void){
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_I2C1_Init();
    lcd_init();
```

```
while (1){
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 1000) == HAL_OK){
        adc_raw = HAL_ADC_GetValue(&hadc1);
        Temp = adc_raw * 3.3 / 4095;
        lcd_put_cursor(0,0);
        lcd_send_string("Temperatura:");
        lcd_put_cursor(0,10);
        lcd_send_string(" ");
        lcd_put_cursor(0,10);
        lcd_send_float(Temp, 1);
        lcd_put_cursor(1,0);
        HAL_Delay(1000); }
}
```



6.6 Problema.

- Tarea: Incorpore al problema del Ciclo de Histéresis con MEF un LCD.

Bibliografia.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media